IT5008: Tutorial 1 — Entity-Relationship Model

Pratik Karmakar

School of Computing,
National University of Singapore

AY26/27 S1

The Book Exchange Scenario

Students at the National University of Ngendipura (NUN) buy books for their studies, and lend/borrow books to/from each other.

Apasaja Private Limited is commissioned by the **NUN Students Association (NUNStA)** to implement an online book exchange system recording:

- Information about **students**,
- **Books** that they own,
- Books that they **lend and borrow**.

What the Database Must Track

- **Student**: name, main department, identified by `email`; `joined` date; `graduation` date *if* graduated.
- **Department**: belongs to *exactly one* faculty; no faculty exists without a department; some departments have no students.
- **Book**: title, author(s), publisher, language, year, edition, `ISBN-10` *and* `ISBN-13`. Books may exist unowned (owner graduated, or advised but not yet bought).
- **Copy**: a student may own several copies of the *same* book, told apart by a **copy number** (consecutive from 1).
- **Loan record**: date a copy was borrowed, date it was returned. A student can only borrow/lend *after* being enrolled.
- **Auditing**: keep books/copies/owners as long as owned by a student *or* referenced by a loan; keep graduated students as long as loan records about their books exist.

Constraints to Capture

Before drawing anything, list out the constraints hiding in the text:

- Student `email` uniquely identifies a student.
- Book `ISBN-10` uniquely identifies a book; so does `ISBN-13`.
- A book may be written by **one or more** authors.
- A department belongs to **exactly one** faculty.
- Graduation date is after the joined date. *(common sense)*
- Return date of a loan is after the borrow date. *(common sense)*
- Borrow date is after the joined date.
- Copy number is consecutive, starting from 1.

Entity Sets

Nouns (not proper nouns) in the text ⇒ entity sets:

- **Students** — "*the name and the main department of each student*"; identified by `email`.
- **Departments** — "*A department in NUN must belong to exactly one faculty*"
- **Faculties** — same quote as above
- **Books** — "*the title, author(s), publisher, . . . for each book*"
- **Author** — "*author(s)*"
- **Copy** — "*We differentiate the copy by its copy number*"
- **Borrow** — "*the date at which a book copy is borrowed . . .*"
- **Graduation** / **Return** — satellite entities holding the *optional* graduation date and loan-return date (see Design Notes).

Relationship Sets

Verbs / possessive phrases $\Rightarrow$ relationship sets:

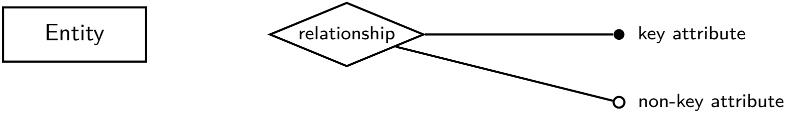
- **enroll** — Students $\leftrightarrow$ Departments: "*main department of each student*"
- **in** — Departments $\leftrightarrow$ Faculties: "*must belong to exactly one faculty*"
- **has** — Graduation $\leftrightarrow$ Students: "*If a student has graduated ...*"
- **own** *(ternary, aggregate)* — Students (*owner*), Books, Copy: "*A student may own multiple copy of the same book*"
- **loan** *(ternary, aggregate)* — Students (*borrower*), **own**, Borrow: "*We refer to this information as a loan record*"
- **been** — **loan** $\leftrightarrow$ Return: "*the date at which it is returned*"
- **by** — Books $\leftrightarrow$ Author: "*author(s)*"

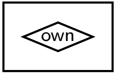
Attributes & Keys

Entity	**Attributes**	**Key**
Students	email, join, name	email
Departments	name	name
Faculties	name	name
Graduation	date	date
Books	isbn10, title (. . .), isbn13	isbn10 *and* isbn13
Author	name	name
Copy	copy num	copy num
Borrow	date	date
Return	date	date

• = key attribute ◦ = non-key attribute (matches the diagram notation)

ER Notation Legend

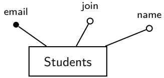




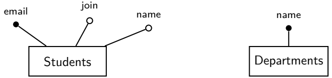
box around a relationship = **aggregation** (it can itself take part in another relationship, e.g. **own** in **loan**, **loan** in **been**)

Rectangle = entity set. Diamond = relationship set. • = key attribute, ∘ = non-key attribute (text sits beside the dot, connected straight to the entity/relationship). Edge labels give (min,max) participation. **own** and **loan** are *ternary* (three participants each).

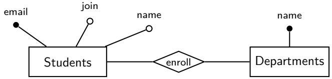
Entity-Relationship Diagram



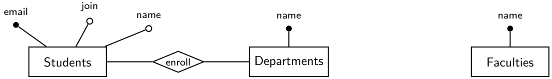
Entity-Relationship Diagram



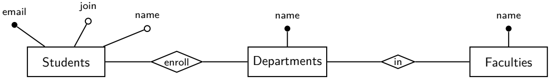
Entity-Relationship Diagram



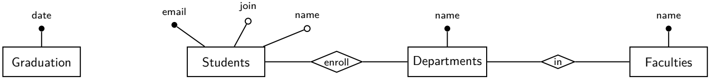
Entity-Relationship Diagram



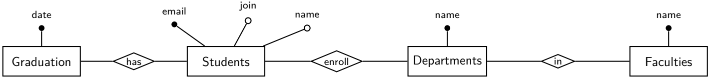
Entity-Relationship Diagram



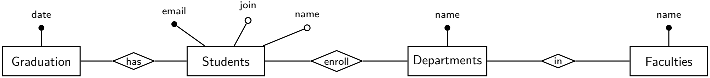
Entity-Relationship Diagram

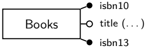


Entity-Relationship Diagram

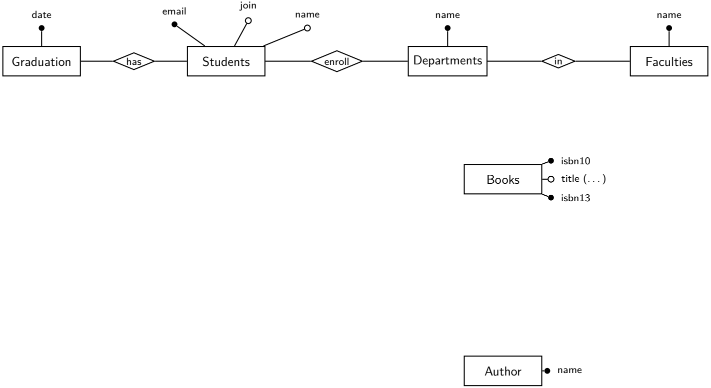


Entity-Relationship Diagram

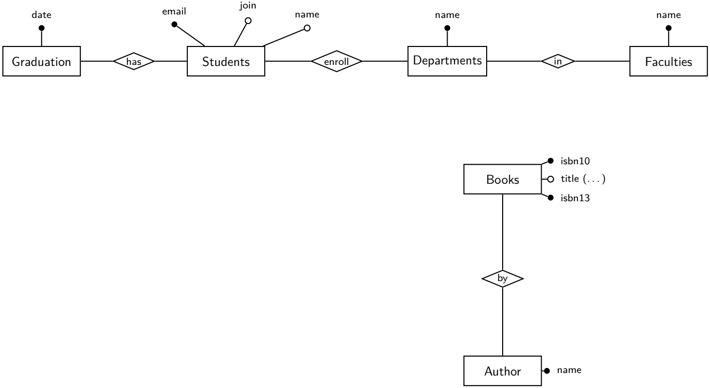




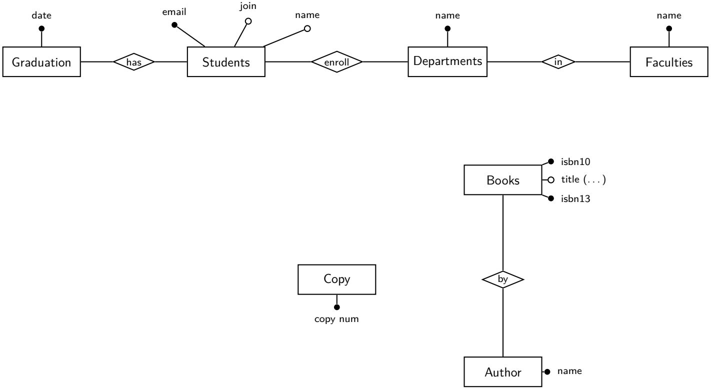
Entity-Relationship Diagram



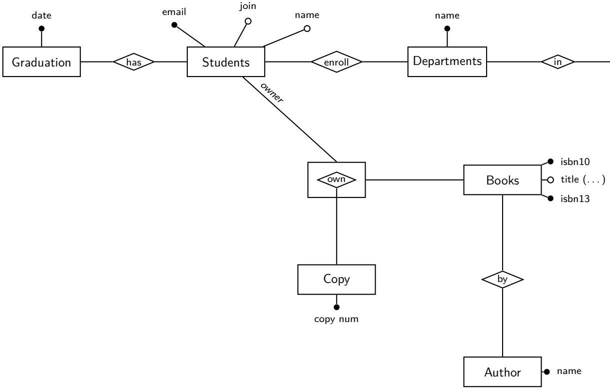
Entity-Relationship Diagram



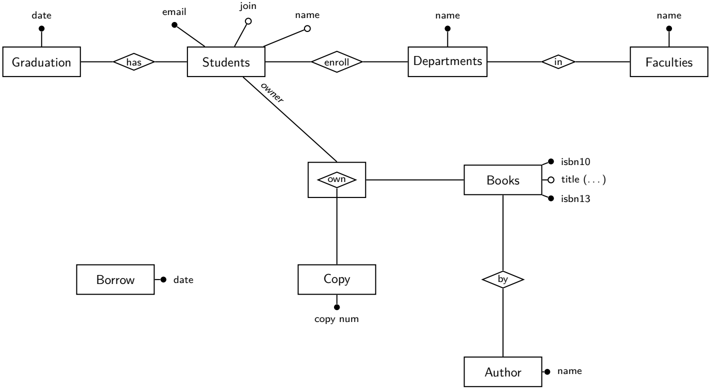
Entity-Relationship Diagram



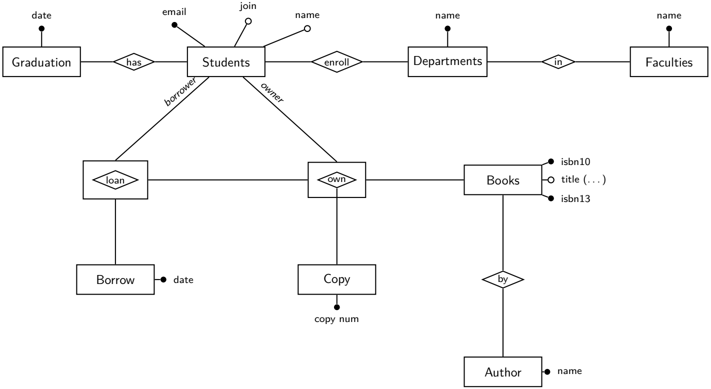
Entity-Relationship Diagram



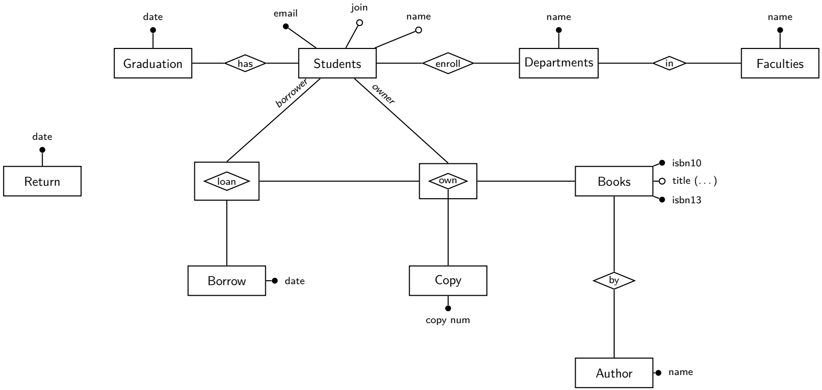
Entity-Relationship Diagram



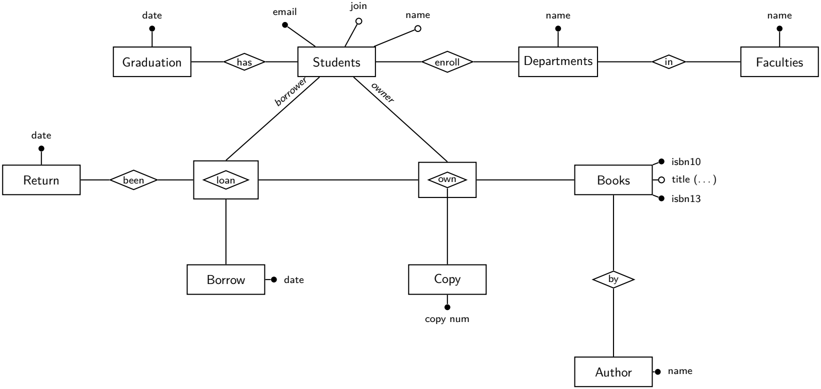
Entity-Relationship Diagram



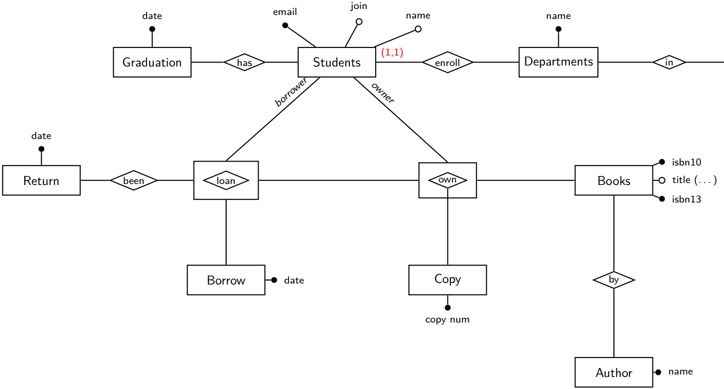
Entity-Relationship Diagram



Entity-Relationship Diagram

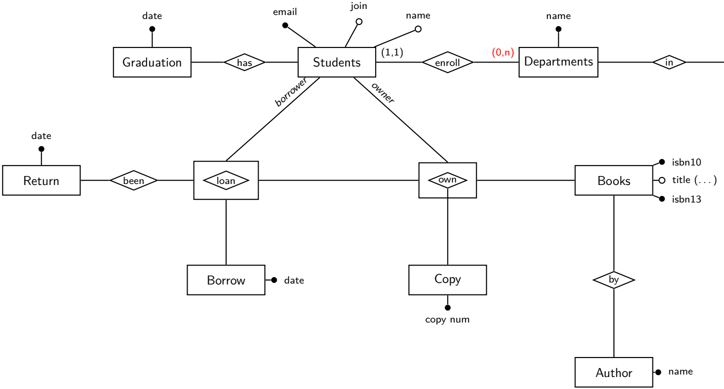


Entity-Relationship Diagram



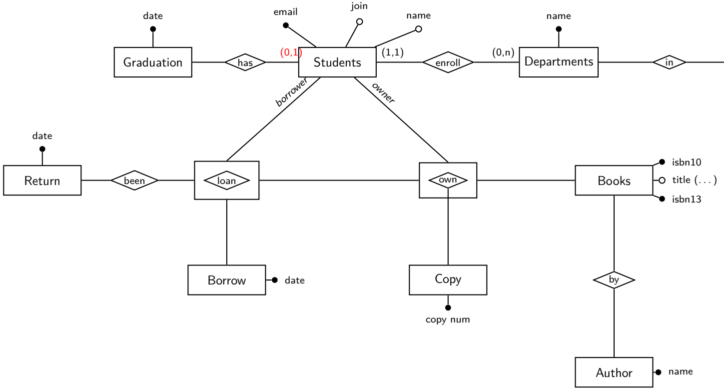
Every student enrolls in exactly one department.

Entity-Relationship Diagram



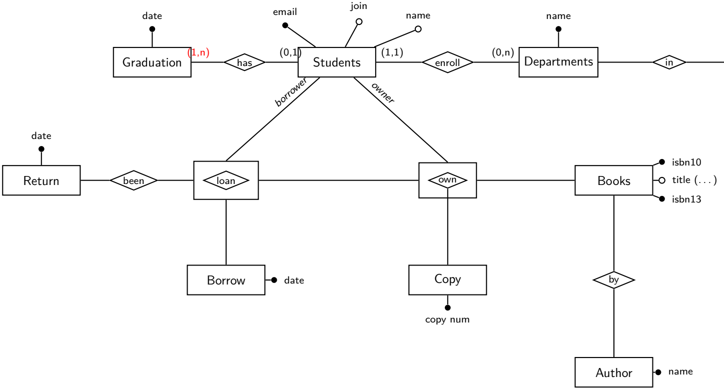
A department may have zero students.

Entity-Relationship Diagram



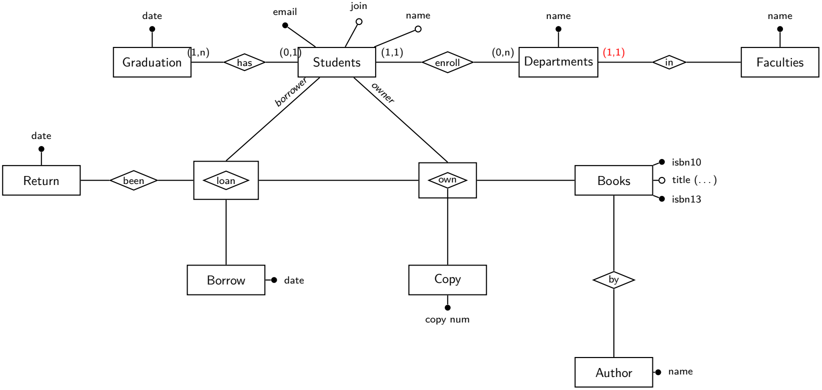
A student may or may not have graduated yet.

Entity-Relationship Diagram



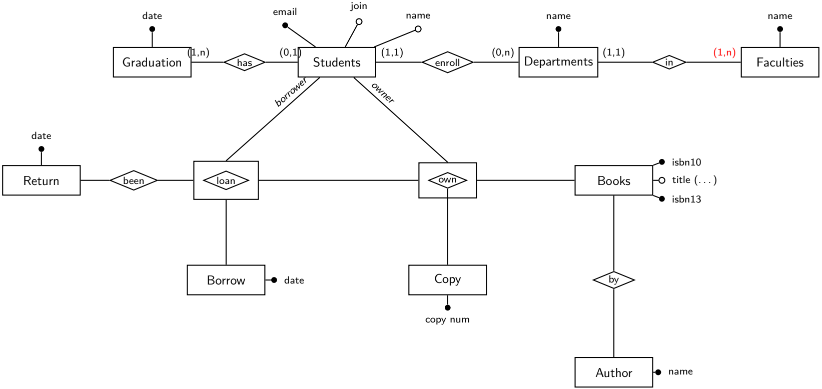
We only store a graduation date once it exists.

Entity-Relationship Diagram



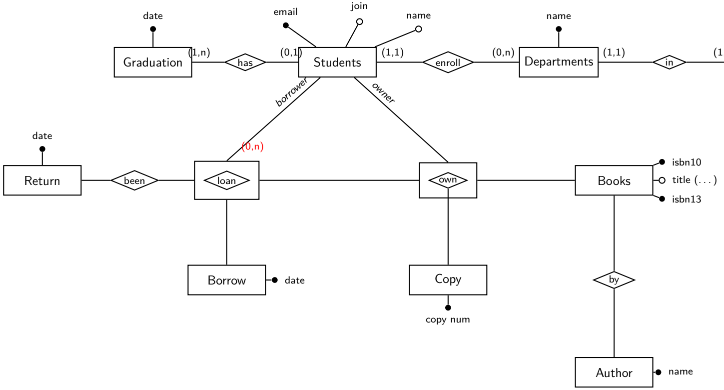
A department belongs to exactly one faculty.

Entity-Relationship Diagram



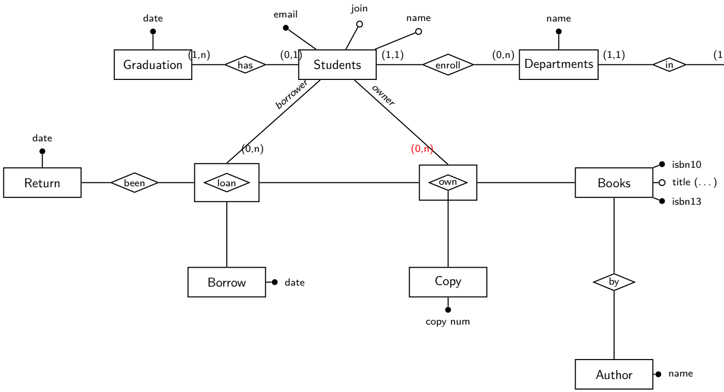
No faculty exists without at least one department.

Entity-Relationship Diagram



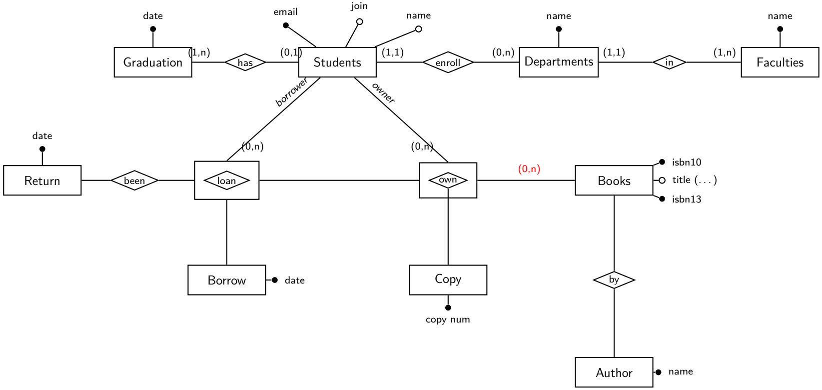
A student may hold zero or many loans.

Entity-Relationship Diagram



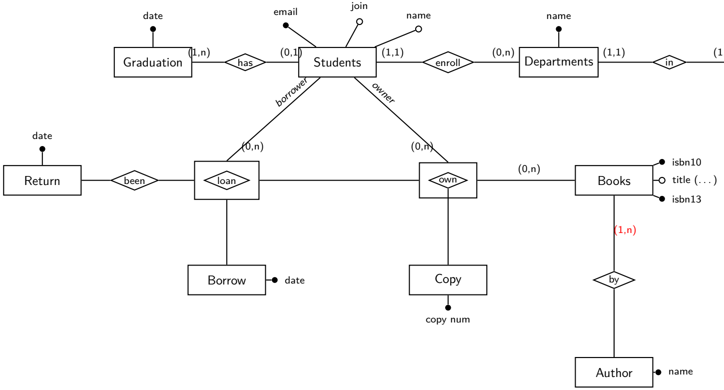
A student may own zero or many copies.

Entity-Relationship Diagram



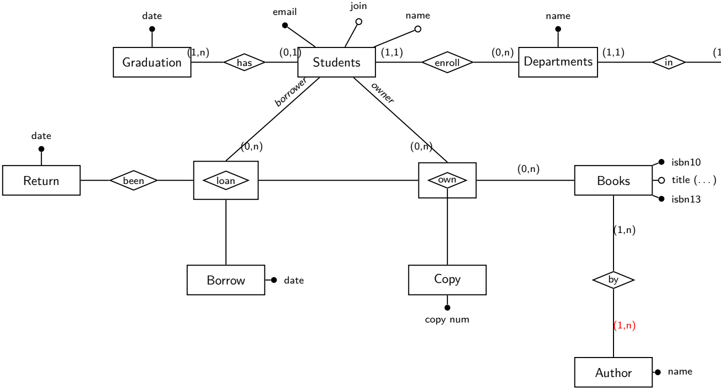
A book may exist with zero owned copies.

Entity-Relationship Diagram



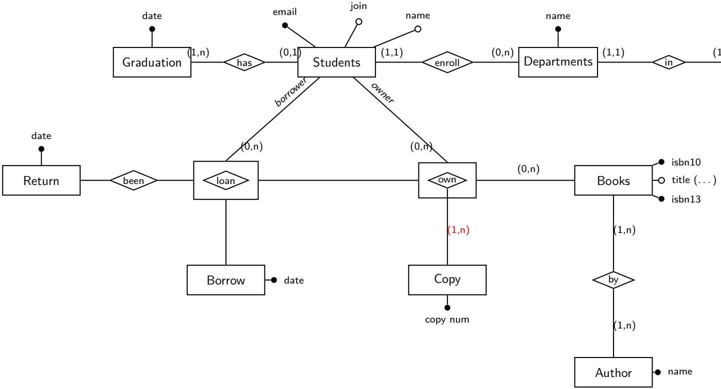
A book is written by one or more authors.

Entity-Relationship Diagram



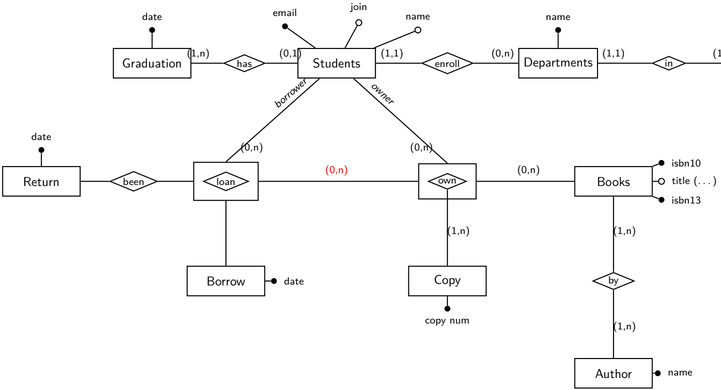
An author is only recorded once they've written a book.

Entity-Relationship Diagram



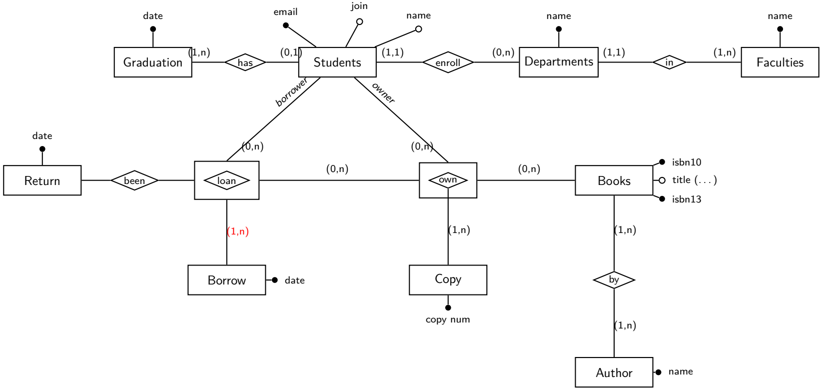
own has no meaning without a copy — forces a merge later.

Entity-Relationship Diagram



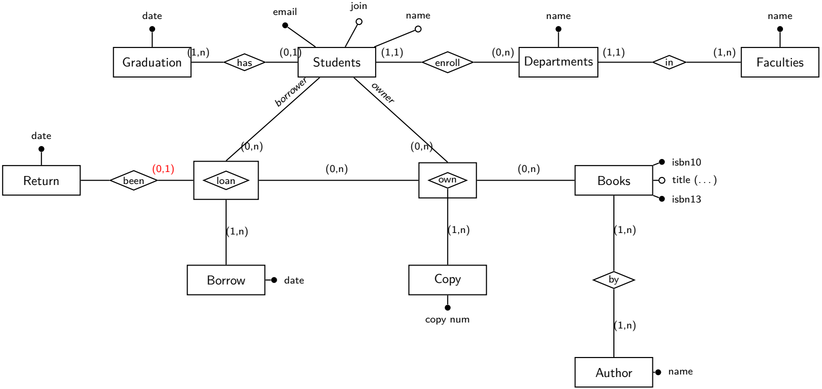
A copy may be loaned zero or many times.

Entity-Relationship Diagram



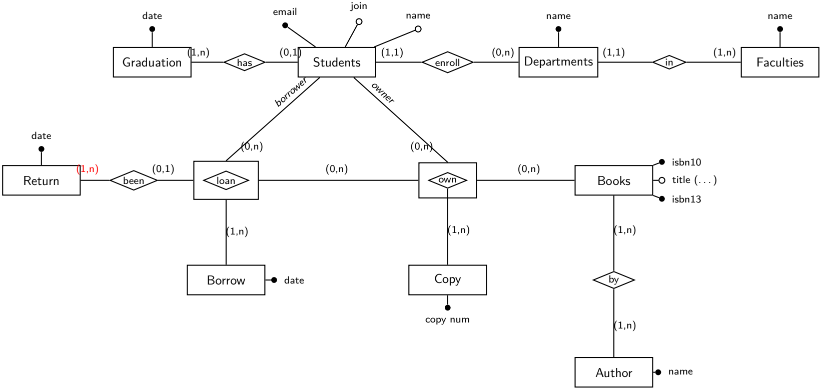
loan has no meaning without a borrow date — forces a merge later.

Entity-Relationship Diagram



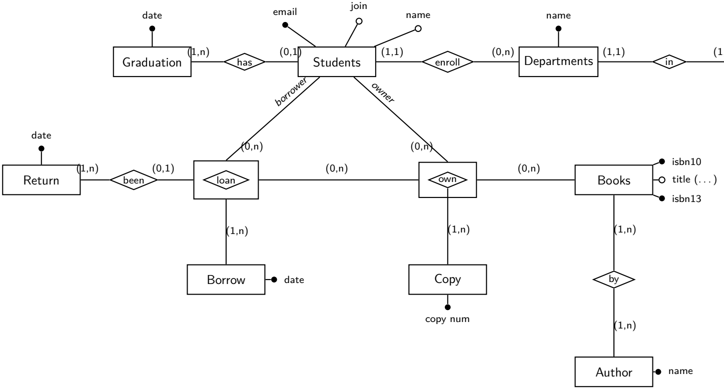
A loan may not yet have been returned.

Entity-Relationship Diagram



We only store a return date once it is actually returned.

Entity-Relationship Diagram



Design Notes — Ternary Relationships & Aggregation

- **own** and **loan** are *ternary*: a student owning/borrowing involves *three* participants at once (student, book/copy, and the copy/borrow event) — a binary relationship cannot express that.
- **Two** relationships must be boxed as **aggregates**, because each is itself a participant in a further relationship (a plain relationship cannot connect directly to another relationship):
  - **own** → boxed, because **loan** needs to reference "this particular owned copy" as a whole, not just `Students` or `Copy` alone.
  - **loan** → also boxed, because **been** needs to reference "this particular loan event" as a whole, not just `Students`, `own`, or `Borrow` alone.
- **Students** plays two *roles*: *owner* (into **own**) and *borrower* (into **loan**) — same entity set, two different participations.

Design Notes — Optional Attributes as Satellite Entities

- **Graduation** → **has** → **Students** is **(1,n)–(0,1)**: a student may or may not have graduated, and we don't want to store an unused graduation date on every student row. Same reasoning for **Return** → **been** → **loan**.
- These **(1,n)** satellite entities (Graduation, Return, and likewise Copy into **own**, Borrow into **loan**) are designed to be *merged* into their relationship at the logical-design stage — see next section.

Relational Schema — Department, Student

```sql
CREATE TABLE department (
  faculty VARCHAR(62) NOT NULL,
  name    VARCHAR(32) PRIMARY KEY
);

CREATE TABLE student (
  name       VARCHAR(32) NOT NULL,
  email      VARCHAR(256) PRIMARY KEY,
  year       DATE NOT NULL,       -- join date
  graduate   DATE CHECK (graduate >= year),
  department VARCHAR(32) NOT NULL
    REFERENCES department(name)
);
```

Deviation: `department–faculty` and `student–Graduation` are each merged into one table (the (1,1) side has no attributes of its own to justify a separate table; merging `graduate` lets us write the `CHECK` in one place).

Relational Schema — Book, Author

```sql
CREATE TABLE book (
  title     VARCHAR(256) NOT NULL,
  publisher VARCHAR(64)  NOT NULL,
  language  VARCHAR(32)  NOT NULL,
  year      DATE NOT NULL,
  edition   INT  NOT NULL,
  isbn10    CHAR(10) NOT NULL UNIQUE,
  isbn13    CHAR(14) PRIMARY KEY
);

CREATE TABLE author (
  name VARCHAR(32),
  book CHAR(14) REFERENCES book(isbn13),
  PRIMARY KEY (name, book)
);
```

Relational Schema — own_copy, loan

```
CREATE TABLE own_copy (
  owner VARCHAR(256) REFERENCES student(email),
  book  CHAR(14) REFERENCES book(isbn13),
  copy  INT CHECK (copy > 0),
  PRIMARY KEY (owner, book, copy)
);

CREATE TABLE loan (
  borrower VARCHAR(256) REFERENCES student(email),
  owner    VARCHAR(256),
  book     CHAR(14),
  copy     INT,
  borrowed DATE,
  returned DATE CHECK (returned >= borrowed),
  FOREIGN KEY (owner, book, copy)
    REFERENCES own_copy(owner, book, copy),
  PRIMARY KEY (borrowed, borrower, owner, book, copy)
);
```

Constraints Not Captured by the DDL

- **Copy number is consecutive from 1** — needs the values across *all* rows for an (owner, book); a single-row `CHECK` cannot see other rows.
- **Borrow date is after the joined date** — compares columns across `loan` and `student`; a single-row `CHECK` cannot reach into another table.
- `graduate >= year` and `returned >= borrowed` **are** captured, via same-row `CHECK` constraints.
- **Audit retention** (keep books/copies/students while referenced by a copy or loan) is captured *implicitly*: the default foreign-key behaviour (`ON DELETE RESTRICT`) blocks any `DELETE` that would orphan an `own_copy` or `loan` row.

Questions?

Drop a mail at: pratik.karmakar@u.nus.edu